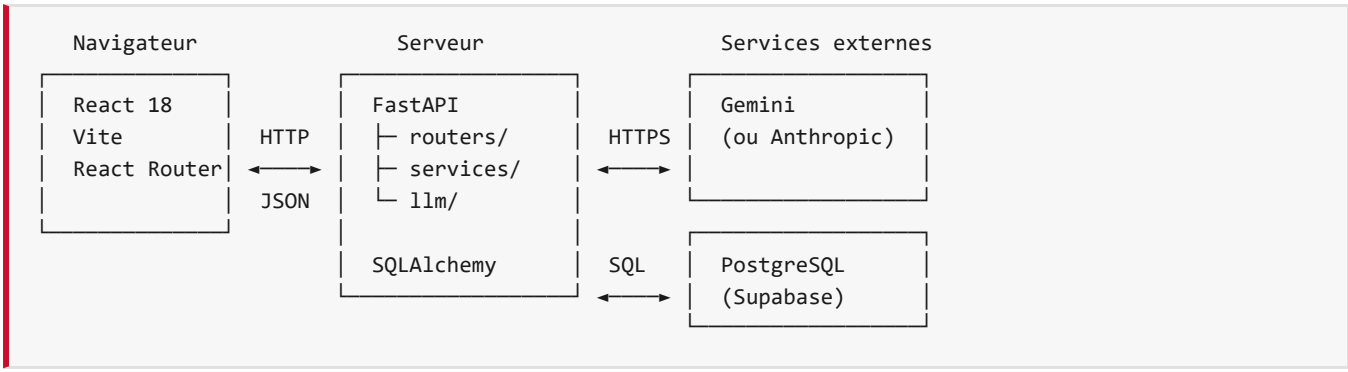
Architecture

1. Vue d'ensemble

Trois tiers classiques, séparés par le réseau :



Le navigateur ne parle **jamais** au fournisseur de modèle. La clé d'API ne quitte pas le serveur, et le masquage des données ne peut pas être contourné depuis le client.

2. Pile technique

Couche	Choix	Version
Front	React + Vite + TypeScript	18.3 / 5.4 / 5.6
Routage	React Router	6.26
Back	Python + FastAPI	3.12+ / 0.143
Validation	Pydantic	2.14
ORM	SQLAlchemy	2.1
Migrations	Alembic	1.20
Base	PostgreSQL (Supabase)	15
Pilote	psycopg	3.3
Appels HTTP	httpx	0.28
Extraction PDF	PyMuPDF	1.28
Tests	pytest	9.1

Les raisons de ces choix, et les alternatives écartées, sont dans `DECISIONS.md`. En résumé : un seul langage côté serveur, la documentation OpenAPI produite automatiquement, et une extraction PDF qui respecte l'ordre de lecture.

3. Organisation du code

```
server/app/
├── main.py              application, CORS, gestionnaires d'erreurs
├── config.py            configuration validée au démarrage
├── db.py                moteur et sessions
├── models.py            7 tables SQLAlchemy
├── errors.py            taxonomie d'erreurs
├── security.py          mots de passe, jetons, dépendance d'authentification
│
├── routers/             auth.py · courses.py · study.py
├── services/
│   ├── study.py         logique métier des trois modules
│   ├── segmentation.py  découpage et sélection des segments
│   └── pdf.py           extraction du texte
├── schemas/
│   ├── api.py           contrats HTTP (camelCase en sortie)
│   └── llm.py           sorties attendues des modules IA
├── prompts/             les prompts versionnés
└── llm/
    ├── pipeline.py      ★ SEULE entrée publique du module
    ├── redact.py        anonymisation et restauration
    ├── run_task.py      appel + validation + relance
    ├── client.py        ⚠ ne pas importer hors de llm/
    └── providers/       gemini.py · anthropic.py

client/src/
├── App.tsx              routage et en-tête
├── pages/               AuthPages · CoursPage · CoursDetailPage
├── components/          ui.tsx · ResumePanel · QuizPanel · ChatPanel
└── lib/                 api.ts · auth.tsx · types.ts
```

4. Règles de dépendance

Elles ne sont pas décoratives : deux d'entre elles portent des garanties de sécurité.

1. **Une route n'appelle jamais `llm/` directement.** Elle passe par `services/study.py`, qui passe par `llm/pipeline.py`.
2. **`llm/client.py` n'est importé nulle part hors de `llm/`.** C'est ce qui rend impossible un appel au modèle sans anonymisation : il n'existe aucun autre chemin.
3. Un service ne connaît ni `Request` ni `Response` : il reçoit des valeurs, il rend des objets.
4. Le front ne compose aucun message d'erreur : il lit `error.code` et `lib/api.ts` traduit.

5. Le pipeline IA

C'est le cœur du projet. Même séquence pour les trois modules :

```
texte brut du cours
      │
      ▼
 1. ANONYMISATION          une seule fois, sur le cours entier
      │                    (jamais segment par segment : les jetons
      │                     seraient numérotés indépendamment)
      ▼
 2. DÉCOUPAGE              paragraphes → phrases → coupe forcée
      │                    recouvrement entre segments voisins
      ▼
 3. APPEL(S) AU MODÈLE     1 appel si le cours est court
      │                    n+1 si long (résumé), ≤4 (quiz), 1 (tuteur)
      ▼
 4. VALIDATION             schéma Pydantic — ce qui ne valide pas n'existe pas
      │                    une relance au plus, avec l'erreur exacte
      ▼
 5. RÉHYDRATATION          les jetons redeviennent les valeurs réelles
      │
      ▼
 6. TÉLÉMÉTRIE             modèle, version de prompt, température,
                           jetons consommés, latence, nombre d'appels
```

Pourquoi l'anonymisation précède le découpage

Masquer chaque segment séparément produirait `[NOM_1]` désignant une personne dans un segment et une autre dans le suivant. L'agrégation des résumés partiels mélangerait alors les identités. Le masquage est donc fait une fois, sur le texte entier, avant la découpe.

Traitement des cours longs

Module	Stratégie	Appels
Résumé	un résumé par segment, puis un appel d'agrégation	n + 1
Quiz	segments échelonnés sur tout le cours, questions réparties	≤ 4
Tuteur	les segments les plus proches de la question (BM25)	1

Mesures réelles du 12 octobre : Atelier 5 (6 229 car.) → 1 segment, 1 appel, 4,1 s. Cours Prompt Engineering (26 040 car.) → 4 segments, 5 appels, 22,4 s.

6. Flux d'une requête

Exemple : l'étudiant demande un résumé.

1. Le front envoie `POST /api/courses/{id}/summary` avec le jeton JWT.
2. FastAPI valide le corps contre `ResumeEntree`.
3. `utilisateur_courant` décode le jeton, sinon `UNAUTHORIZED`.
4. `services/study.resumer` vérifie la **propriété** du cours, sinon `FORBIDDEN`.
5. Cache : un résumé du même cours au même niveau existe-t-il ? Si oui, il est renvoyé sans appel au modèle.
6. Sinon `llm/pipeline.resumer_cours` exécute le pipeline ci-dessus.

7. Le résumé, sa télémétrie et la charge envoyée sont enregistrés.
8. Le front affiche la synthèse, la télémétrie, et peut déplier la charge réellement transmise au modèle.

7. Deux niveaux de reprise, à ne pas confondre

C'est une distinction que le rapport doit faire, parce qu'elle est souvent mal comprise.

	Niveau transport (`client.py`)	Niveau schéma (`run_task.py`)
Déclencheur	`LLM_UNAVAILABLE` (503)	sortie non conforme
Tentatives	2 reprises, attentes 800 ms puis 2 500 ms	1 relance
Ce qui change	rien, on refait le même appel	le prompt reçoit l'erreur exacte
Ne reprend pas	clé refusée, modèle inexistant, quota, délai	—

8. Que se passe-t-il si…

Panne	Comportement
Clé absente	`LLM_KEY_MISSING` (503) — le serveur démarre quand même et le prévient dans ses journaux
Délai dépassé	`LLM_TIMEOUT` (504), pas de reprise
Quota épuisé	`LLM_QUOTA_EXCEEDED` (429), pas de reprise
Modèle retiré	`LLM_UNAVAILABLE` (502), **en nommant le modèle**
Sortie non conforme 2 fois	`LLM_INVALID_OUTPUT` (502), rien n'est stocké
Base injoignable	`INTERNAL` (500), détails journalisés, pas exposés
Échec en plein chat	la question de l'étudiant reste enregistrée, le fil est intact

9. Performance

Le temps est dominé par l'appel au modèle (2 à 6 s), pas par le code. D'où :

- des routes `def` et non `async def` — FastAPI les exécute dans un pool de fils, ce qui suffit largement à notre échelle et évite les pièges du pilote asynchrone avec le pooler de Supabase ;
- un cache des résumés, qui supprime l'appel plutôt que de l'accélérer ;
- un nombre d'appels plafonné par module.

10. Ce que l'architecture ne couvre pas

À dire plutôt qu'à masquer :

- pas de limitation de débit par utilisateur (`RATE_LIMITED` existe dans la taxonomie mais n'est émis par aucune route) ;
- pas de tâches de fond : un résumé de cours long bloque la requête 20 s ;
- pas de métriques agrégées ni de supervision ;
- la table `benchmark_runs` existe mais n'est pas encore alimentée.